Обработка исключения

try.. exept..

Обработка исключений в Python

Программа, написанная на языке Python, останавливается сразу как обнаружит ошибку. Ошибки могут быть:

Синтаксические ошибки — возникают, когда написанное выражение не соответствует правилам языка (например, написана лишняя скобка);

Логические ошибки — это ошибки, когда синтаксис действительно правильный, но логика не та, какую вы предполагали. Программа работает успешно, но даёт неверные результаты.

Исключения — возникают во время выполнения(runtime) программы (например, при делении на ноль).


wikiHow to Catch a Grasshopper

Перехват ошибок во время выполнения

Не всегда при написании программы можно сказать возникнет или нет в данном месте исключение. Чтобы приложение продолжило работу при возникновении проблем, такие ошибки нужно перехватывать и обрабатывать с помощью ловуши **try/except**.

В каких случаях нужно предусматривать обработку ошибок ?

- **Работа с файлами** (нет прав доступа , диск переполнен и т.д)
- **Работа с СУБД** (сервер не доступен, ошибка выполнения запроса и т.д)
- **Работа с сетью** (сеть недоступна, ошибка соединения, обработка кода возврата и т.д)
- **Работа с различными библиотеками** которые бросают exeption в случае обнаружения ошибки.
- **Работа с критическими участками кода**

Как устроен механизм исключений ?

В Python есть встроенные исключения, которые появляются после того как интерпретатор находит ошибку. В этом случае текущий процесс временно приостанавливается и передает ошибку на уровень вверх до тех пор, пока она не будет обработана. В случае когда ошибка не обработывается, программа прекратит свою работу и в консоли мы увидим **Traceback** с подробным описанием ошибки.

Иерархия классов исключений Python

https://docs.python.org/3/library/exceptions.html

```
BaseException
 +-- SystemExit
 +-- KeyboardInterrupt
 +-- GeneratorExit
 +-- Exception
      +-- StopIteration
      +-- StopAsyncIteration
      +-- ArithmeticError
      |    +-- FloatingPointError
      |    +-- OverflowError
      |    +-- ZeroDivisionError
      +-- AssertionError
      +-- AttributeError
      +-- BufferError
      +-- EOFError
      +-- ImportError
      |    +-- ModuleNotFoundError
      +-- LookupError
      |    +-- IndexError
      |    +-- KeyError
      +-- MemoryError
      +-- NameError
      |    +-- UnboundLocalError
      +-- OSError
```

BaseException – базовое исключение, от которого берут начало все остальные.
SystemExit – исключение, порождаемое функцией sys.exit при выходе из программы.
KeyboardInterrupt – порождается при прерывании программы пользователем
GeneratorExit – порождается при вызове метода close объекта generator.
Exception – а вот тут уже заканчиваются полностью системные исключения (которые лучше не трогать) и начинаются обыкновенные, с которыми можно работать.

StopIteration – порождается встроенной функцией next, если в итераторе больше нет элементов.

ArithmeticError – арифметическая ошибка.

FloatingPointError – порождается при неудачном выполнении операции с плавающей запятой. На практике встречается нечасто.
OverflowError – возникает, когда результат арифметической операции слишком велик для представления.
ZeroDivisionError – деление на ноль.

Генерация исключения

Напишем код, который будет создавать исключительную ситуацию. К примеру, попробуем поделить число на 0:

```
>>> print(1 / 0)
```

В командной оболочке получим следующее:

```
Traceback (most recent call last):
  File "", line 1, in
  ZeroDivisionError: division by zero
```

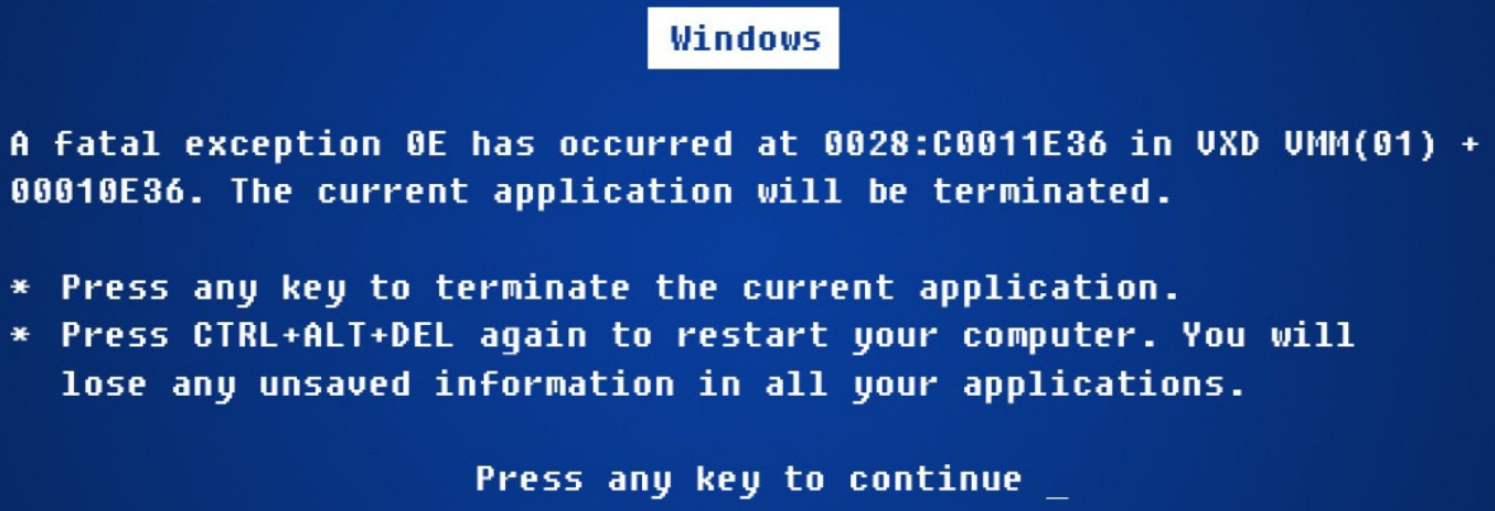
Разберём это сообщение подробнее:

Интерпретатор нам сообщает о том, что он поймал исключение и напечатал информацию: Traceback (most recent call last).

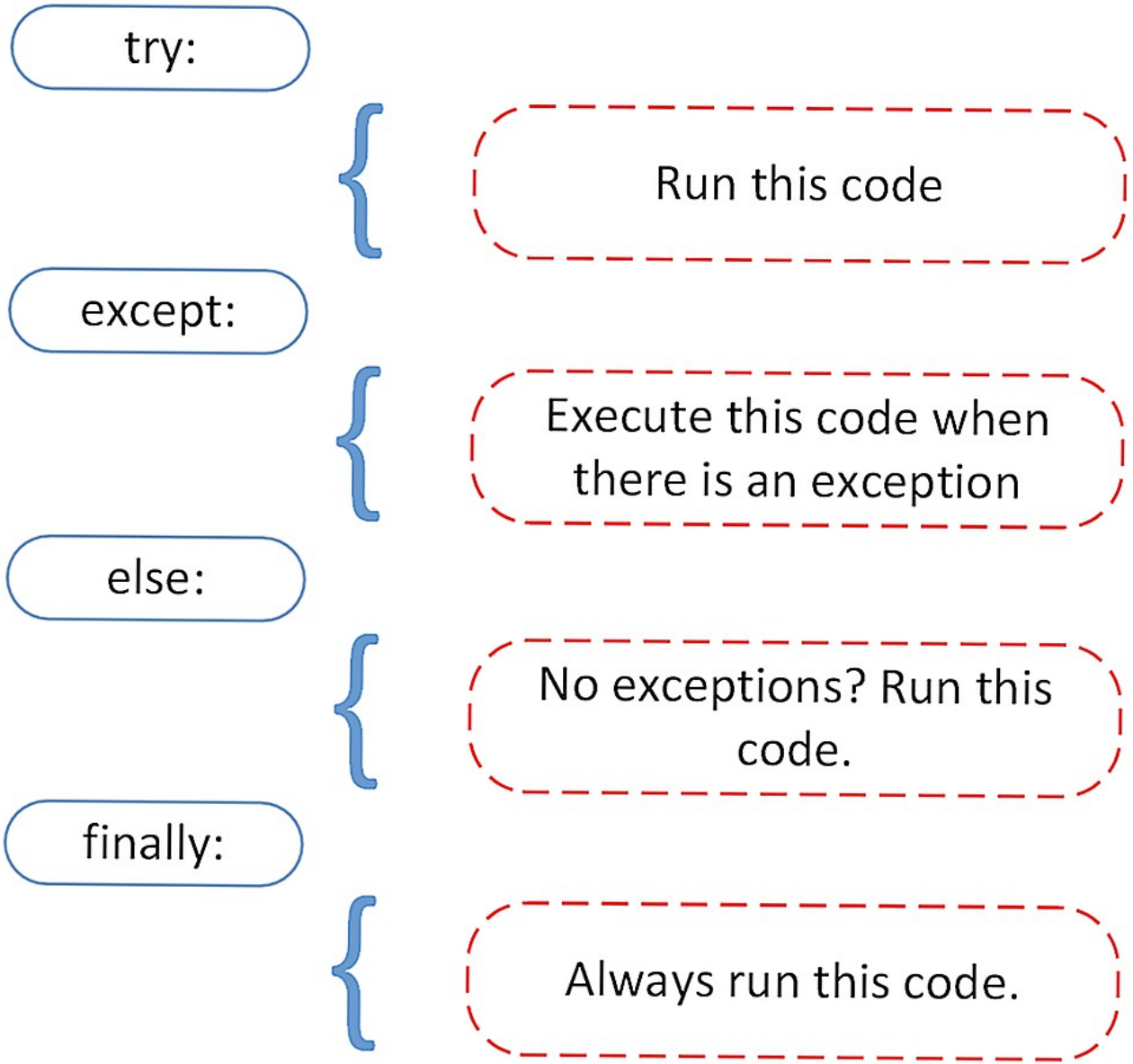
Далее имя файла File "". Имя пустое, потому что этот код был запущен в интерактивном интерпретаторе, строка в файле line 1;

Название исключения **ZeroDivisionError** и краткое описание исключения division by zero.

При выбросе исключения программа закрывается и не выполняет код, который следует за строкой, в которой произошло исключение!



Синтаксис конструкции



Пример

```
try:
    a = 1/0

except ZeroDivisionError:
   print("Возникло исключение:ошибка деления на ноль! ")
   a = 0

else:
    print("Ветка else вызывается если не возникло
исключения")

finally:
    print("Аккуратно обрабатываем ошибку и идем дальше...")
```

Ловушка для двух исключений

```
from calc import div


try:
    result = div(100, 0)
    print("Расчёт проведён успешно")
except (ZeroDivisionError, KeyError) as e:
    print("Ошибка деления или ошибка обращения по
          ключу. Вот она:", e)
print(result)
```

Несколько ловушек

```
from calc import div
try:
    result = div(100, 0)
    print("Расчёт проведён успешно")
except ZeroDivisionError as e:
    print("Ошибка деления произошла", e)
except KeyError as e:
    print("Ошибка обращения по ключу произошла:", e)

print(result)
```

Перехват ошибки чтения

```
try:
    file = open('ok123.txt', 'r')
except FileNotFoundError as e:
    print(e)
```

```
> [Errno 2] No such file or directory: 'ok123.txt'
```

Порядок следования обработки

```
try:
    file = open('ok123.txt', 'r')
except Exception as e:
    print(Exception ,e)
except FileNotFoundError as e:
    print(FileNotFoundError, e)
```

Какой блок поймает исключение если файл не обнаружен ?

Обратите внимание, что:

- Срабатывает не более одного оператора except,
- Сработает первое совпавшее предложение except.

Блок `finally` выполняется всегда

```
try:
    file = open('ok.txt', 'r')
    lines = file.readlines()
    print(lines[5]) ← Обращение к несуществ. строке
except IndexError as e:
    print(e)
finally:
    file.close()
    if file.closed:
        print("файл закрыт!")
> файл закрыт!
```

Каскадное включение перехватчиков

```
try:
    ……
        try:
            ……
        except e:
            ……
except e:
    ……
```

```
import numpy as np

def divide(x, y):
    try:
        out = x/y
    except:
        try:
            out = np.inf * x / abs(x)
        except:
            out = np.nan
    finally:
        return out

divide(15, 3)  # 5.0
divide(15, 0)  # inf
divide(-15, 0) # -inf
divide(0, 0)   # nan
```

Генерация исключений в Python

Для принудительной генерации исключения используется инструкция **raise.**

```
try:
    raise Exception("Что то пошло не так")
except Exception as e:
    print("Message:" + str(e))
```

Валидатор входного строкового значения на имя человека.

```
def validate(name):
    if len(name) < 10:
        raise ValueError


try:
    name = input("Введите имя:")
    validate(name)
except ValueError:
    print("Имя слишком короткое:")
```

Пользовательские исключения

В Python можно создавать собственные исключения. Такая практика позволяет увеличить гибкость процесса обработки ошибок в рамках той предметной области, для которой написана ваша программа.

```
class NameTooShortError(ValueError):
    pass

def validate(name):
    if len(name) < 10:
        raise NameTooShortError
```

Пользовательские исключения в Python

Для реализации собственного типа исключения необходимо создать класс, являющийся наследником от одного из классов исключений.

```
class NegValException(Exception):
    pass

try:
    val = int(input("input positive number: "))
    if val < 0:
        raise NegValException("Neg val: " + str(val))
    print(val + 10)
except NegValException as e:
    print(e)
```

Вызов конструктора базового класса

```
class NegValException(Exception):
    def __init__(self, number):
        super().__init__(f"Neg val:  {number}")
        self.number = number


try:
   val = int(input("input positive number: "))
   if val < 0:
       raise NegValException(val)

except NegValException as e:
  print(e)
```

Оператор try/except* со звездочкой.

```
# Новое в Python 3.11
Можно использовать несколько специальных операторов
except*, каждое из которых будет обрабатывать часть
группы исключений.

try:
    raise ExceptionGroup("eg",
        [ValueError(1), TypeError(2), OSError(3),
OSError(4)])

except* TypeError as e:
    print(f'caught {type(e)} with nested
{e.exceptions}')

except* OSError as e:
    print(f'caught {type(e)} with nested
{e.exceptions}')
```

- # Обратите внимание, что:
- # Срабатывает не более одного оператора except,
- # Сработает первое совпавшее предложение except.



ООП в Python

Часть III

Типы методов

В Python есть два типа функций, которые могут быть созданы в классе:

- Статические методы.
- Методы класса.



@classmethod

and

@staticmethod

Статический метод

- Может быть определен только внутри класса, но не для объектов класса.
- Его можно вызвать непосредственно из класса по ссылке на имя класса.
- Он не может получить доступ к атрибутам класса
- Статический метод связан с классом. Таким образом, он не может изменить состояние объекта.
- Он также используется для разделения служебных методов для класса.
- Все объекты класса используют только одну копию статического метода.

Есть два способа определить статический метод в Python:
Использование метода **staticmethod()**
Использование декоратора **@staticmethod.**

Определение Static Method in Python

```
#Перед реализацией метода нужно добавить декоратор
@staticmethod

class C:
    @staticmethod
    def f(arg1, arg2, ...): …

# После объявления класса сделать обычный метод
статическим
class Employee:
    def sample(x):
        print('Inside static method', x)

# convert to static method
Employee.sample = staticmethod(Employee.sample)
# call static method
Employee.sample(10)
```

Вызов через класс

```
class DB:
# Определяем статический метод использовав декоратор
    @staticmethod
    def get_conn():
        print("Получим дискриптор соединения с БД!")


# Вызов метода
conn = DB.get_conn()
```

Вызов через объект

```
class DB:
# Определяем статический метод использовав декоратор
    @staticmethod
    def get_conn():
        print("Получим дискриптор соединения с БД!")


test_sysytem_db = DB()
test_sysytem_db.get_conn()
```

При таком вызове не происходит подкапотной передачи self. А значит нет доступа к атибутам объекта.

А можно как то это исправить ?

Учим статический метод работать с экземляром класса

```
class DB:

    def __init__(self):
        self.name = "TestSystem"


    @staticmethod
    # Определяем переменную которая будет принимать объект
    def get_conn(self_):

        print(f"Получим дискриптор соединения с БД
{self_.name}!")

    # Вызов метода


test_sysytem_db = DB()
test_sysytem_db.get_conn(test_sysytem_db)
```

Вызов статического метода из обычного

```
class DB:
# Определяем статический метод использовав декоратор
    @staticmethod
    def get_conn():
        print("Получим дискриптор соединения с БД!")
    def get_session(self):
        #вызов статического метода
        conn = DB.get_conn()
        conn = self.get_conn()
        ……
test_sysytem_db = DB()
test_sysytem_db.get_session()
```

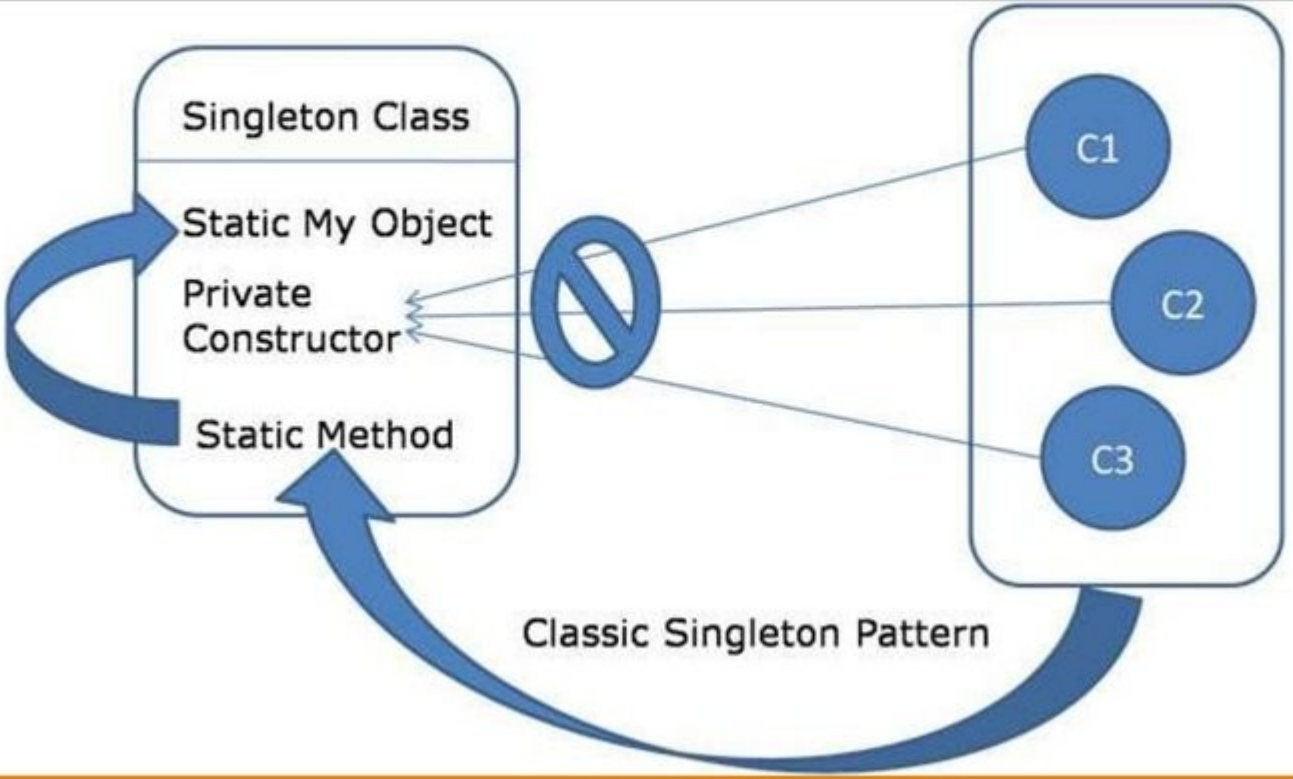


Для чего мы используем static methods ?

Паттерн Singleton

Синглтон (одиночка) – это паттерн проектирования, цель которого ограничить возможность создания объектов данного класса одним экземпляром. Он обеспечивает глобальность до одного экземпляра и глобальный доступ к созданному объекту.

Паттерн Singleton для подключения бд



```
class DB:
    __instance__ = None

    def __init__(self):
        # Проверяем конструктор на сущ. экземпляр
        if DB.__instance__ is None:
            DB.__instance__ = self
        else:
            raise Exception("We can not creat another class")

    @staticmethod
    def get_instance():
        # We define the static method to fetch instance
        if not DB.__instance__:
            DB()
        return DB.__instance__


mongo = DB()
print(mongo)

my_db = DB.get_instance()
print(my_db)

another_db = DB.get_instance()
print(another_db)

new_gover = DB() ← Что будет при вызове ?
```

Методы конструирования и инициализации __new__ __init__

```
class Person(object):
    instance = None
    def __new__(cls, *args, **kwargs):
        if not cls.instance:
            cls.instance = object.__new__(cls)
        return cls.instance
    def __init__(self):
        self.__name = 'Peter I'

obj_one = Person()
obj_two = Person()
print( obj_one is obj_two ) ?
#True
```

Метод **__new__** вызывается первым. Он открывает пространство памяти затем вызывается метод __init__ . Используется для переопределения immutable классов (int, str, tuple)

Методы класса

@classmethod — это метод, который получает класс в качестве неявного первого аргумента, точно так же, как обычный метод экземпляра получает экземпляр. Это означает, что вы можете использовать класс и его свойства внутри этого метода, а не конкретного экземпляра.

Метод класса

- Может быть определен только внутри класса
- Получает класс в качестве неявного первого аргумента
- Его можно вызвать непосредственно из класса по ссылке на имя класса.
- Он не может получить доступ к атрибутам класса
- Не может изменить состояние объекта.
- Все объекты класса используют только одну копию метода класса.

Есть два способа определить статический метод в Python:
Использование метода **classmethod()**
Использование декоратора **@classmethod.**

Объявление метода

```
class MyClass():
    TOTAL_OBJECTS = 0
    def __init__(self):
        MyClass.TOTAL_OBJECTS = MyClass.TOTAL_OBJECTS + 1

    @classmethod
    def total_objects(cls):
        print("Total objects: ", cls.TOTAL_OBJECTS)

# Вызов через объект
my_obj1 = MyClass()
my_obj1.total_objects()
# Вызов через класс
MyClass.total_objects() ← что вернет вызов ?
```

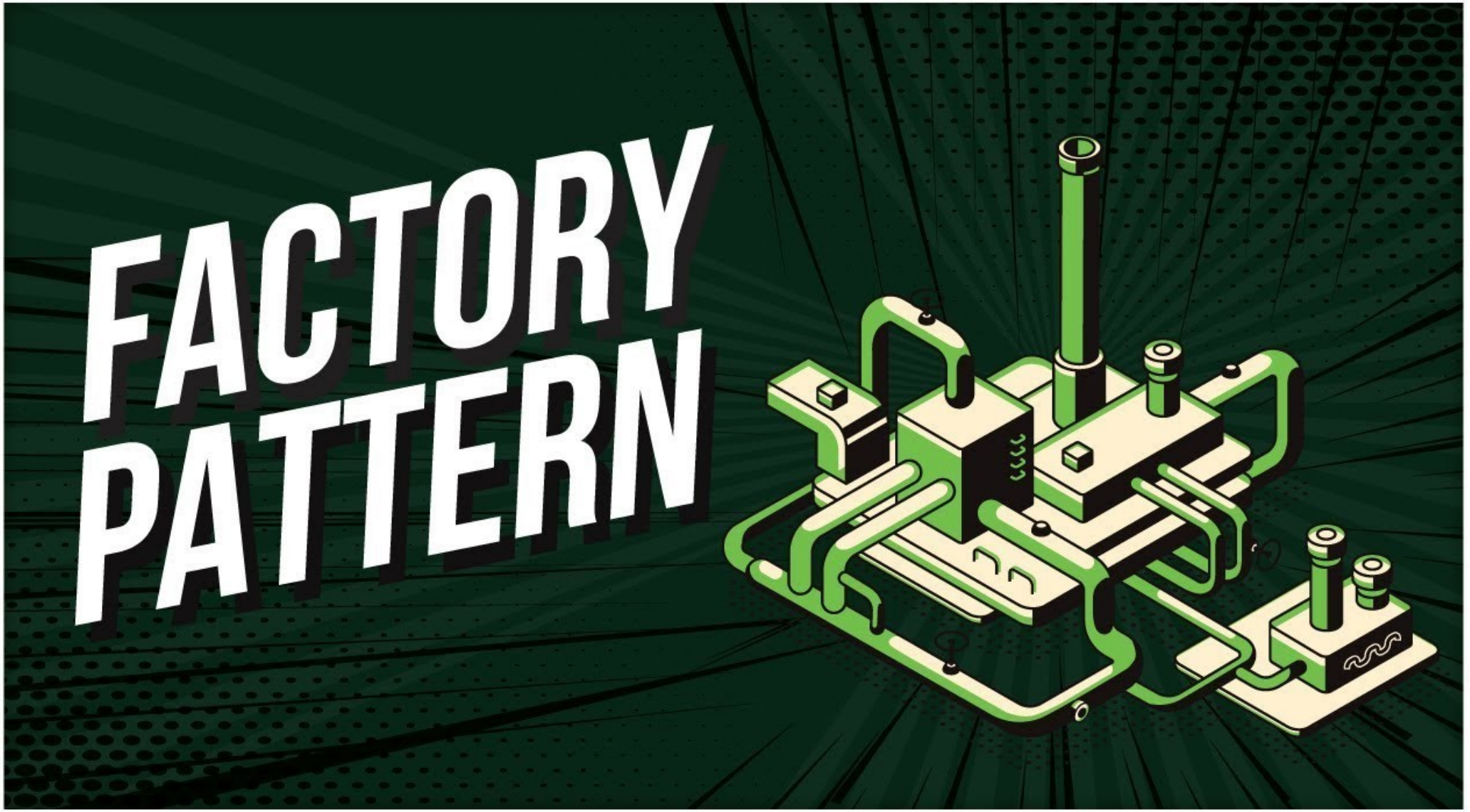
Объявление метода

```
class Coffee:
  def __init__(self, milk, beans):
    self.milk = milk # percentage
    self.coffee = 100 - milk
    self.beans = beans

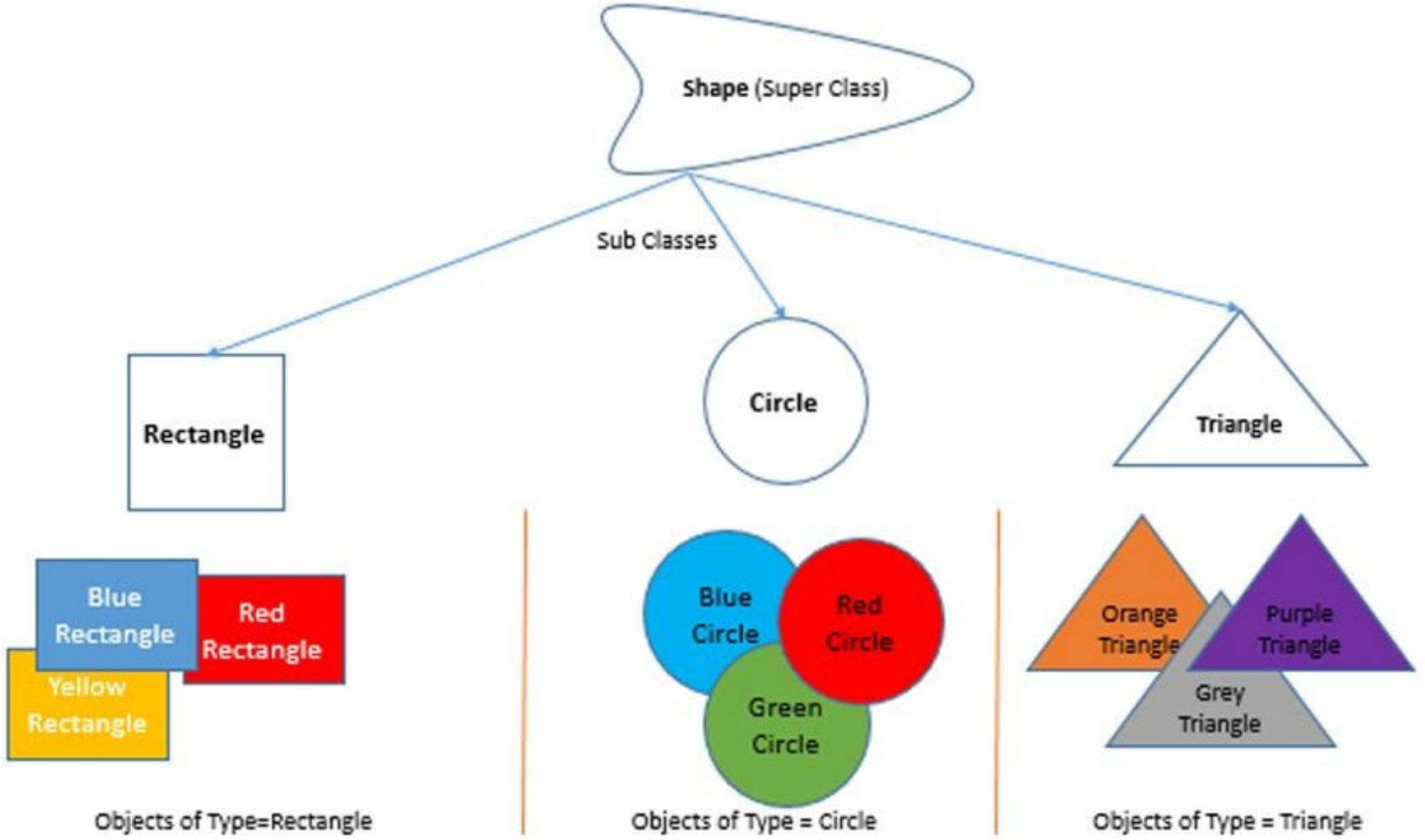
  def __repr__(self):
    return f'Milk={self.milk}% Coffee={self.coffee}%
        Beans={self.beans}'
  def cappuccino(cls):
    return cls(80, 'Arrabica')

Coffee.cappuccino = classmethod(Coffee.cappuccino)
print(Coffee.cappuccino())
```

Шаблон проектирования “Фабрика



Class Object in Python



```
# Рассмотрим фабрику
class Shape:
    def draw(self):
        raise NotImplementedError('This method should
        have implemented.')

class Triangle(Shape):
     def draw(self):
          print("треугольник")

class Rectangle(Shape):
    def draw(self):
          print("прямоугольник")

class ShapeFactory:
    def getShape(self, shapeType):
          if shapeType == 'Triangle':
              return Triangle()
          elif shapeType == 'Rectangle':
              return Rectangle()
          else:
              pass
obj = ShapeFactory()
trgl =  obj.getShape("Triangle")
trgl.draw()
```

```
class Coffee:
  def __init__(self, milk, beans):
    self.milk = milk # percentage
    self.coffee = 100-milk # percentage
    self.beans = beans
  def __repr__(self):
    return f'Milk={self.milk}% Coffee={self.coffee}%
                    Beans={self.beans}'
  @classmethod
  def cappuccino(cls):
    return clf(80, 'Arrabica')

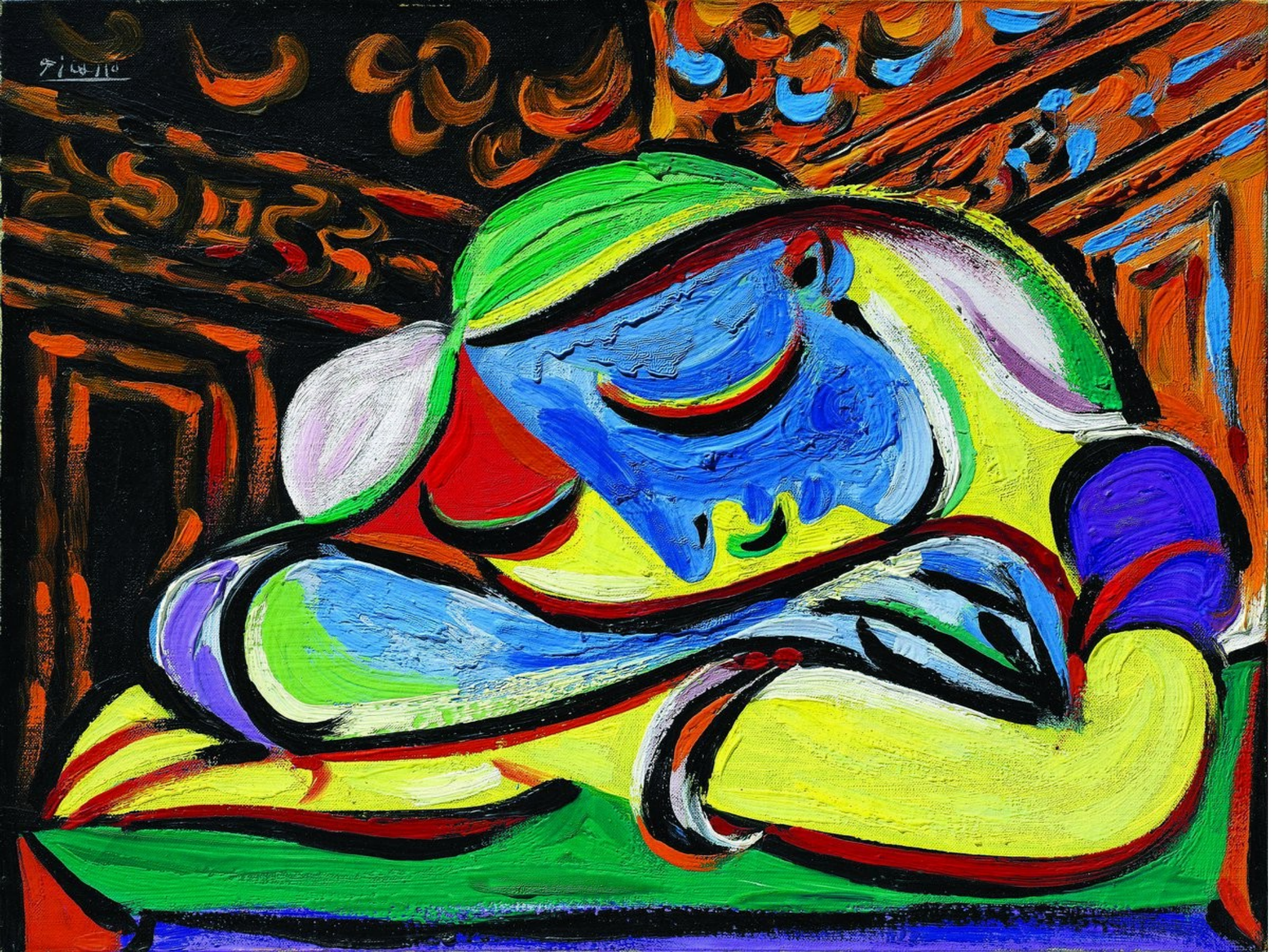
  @classmethod
  def espresso_macchiato(cls):
    return cls(30, 'Robusta')

  @classmethod
  def latte(cls):
    return cls(95, 'Arrabica')

print(Coffee.cappuccino())
print(Coffee.espresso_macchiato())
print(Coffee.latte())
```

Заключение

Декоратор аннотации @classmethod используется для создания фабричных методов, поскольку они могут принимать любой ввод и предоставлять объект класса на основе параметров и обработки.



Абстрактные классы и методы в Python

- **Абстрактные классы** – реализуют механизм организации объектов в иерархии, позволяющий утверждать о наличии требуемых методов.

- **Абстрактный метод** – это метод для которого отсутствует реализация. Объявляется с помощью декоратора @abstractmethod из модуля abc

.

Абстрактные классы и методы в Python

Чтобы объявить абстрактный класс, нам сначала нужно импортировать модуль abc . Давайте посмотрим на пример.

```
from abc import ABC
class abs_class(ABC):
        @abstractmethod
        def render(self):
            pass
```

Абстрактный базовый класс – класс, на основе которого

Нельзя создать экземпляр объекта.

Абстрактный метод — это метод, определенный в базовом классе, но он может не обеспечивать какую-либо реализацию

Абстрактный базовый класс – класс, на основе которого нельзя создать экземпляр объекта.

```
from abc import ABC
class AbsClass(ABC):
      @abstractmethod
      def render(self):
          pass

obj = AbsClass() # вызовет ошибку
```

Чтобы объявить абстрактный класс, нам сначала нужно импортировать модуль abc .

```
from abc import ABC, abstractmethod

class Absclass(ABC):
    def print(self,x):
        print("Passed value: ", x)
    @abstractmethod
    def task(self):
        print("We are inside Absclass task")

class test_class(Absclass):
    def task(self):
        print("We are inside test_class task")


test_obj = test_class()
test_obj.task()
test_obj.print("10")
```

Заключение.

Используя концепцию абстрактных классов, мы можем улучшить качество архитектуры приложения, уменьшить объем работы и при этом, обеспечить легкость дальнейшей поддержки кода.

Спасибо за внимание !